Halo 2 Guide

The Halo 2 proof system, from arithmetisation to recursion

m@rek.onl

Abstract

Assuming the mathematics of the *Math Guide* and the cryptographic primitives of the *Crypto Guide*, this volume of *The Zcash Arboretum* develops the Halo 2 proof system: the polynomial-IOP design pattern, a rigorous account of knowledge soundness and the algebraic group model, recursive proof composition and accumulation schemes, and finally Halo 2 itself — PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment and the Halo accumulation trick, and the complete protocol.

Contents

1 Introduction

This is the *Halo 2 Guide*, the third volume of *The Zcash Arboretum*, developing the cryptography behind Zcash's Orchard protocol. The first two volumes — the *Math Guide* and the *Crypto Guide* — constructed, from the ground up, the mathematics (finite fields, elliptic curves, polynomials and evaluation domains, probability) and the cryptographic primitives (hash functions, commitments, Σ-protocols and the Fiat–Shamir transform, zero knowledge, polynomial commitment schemes) that this volume assumes. The present volume assembles those ingredients into a *proof system*.

The development proceeds in four steps. First, we make the polynomial-IOP design pattern precise: the recipe "encode a computation as polynomial identities over an evaluation domain, then compile to a succinct argument with a polynomial commitment." Next, we treat knowledge soundness rigorously, including the algebraic group model and the tree-of-transcripts extractor that the inner-product argument requires. We then motivate recursive proof composition and develop the accumulation schemes that make it practical. Finally, we assemble Halo 2 itself: PLONKish arithmetisation, the permutation and lookup arguments, the inner-product polynomial commitment with a worked example and its accumulation (the "Halo trick"), and the complete seven-phase protocol. The *Ironwood Guide* subsequently uses this proof system to construct a shielded payment protocol.

2 Polynomial IOPs and the SNARK design pattern

The modern SNARK design pattern factors the protocol into two clean layers: a *polynomial interactive oracle proof* (PIOP) for the relation, and a polynomial commitment scheme (the *Crypto Guide*) that realises the PIOP's oracles. This factoring, due in its current form to Bünz, Fisch, Szepieniec, and others, has substantially clarified the nature of SNARKs.

2.1 The polynomial IOP, formally

Definition 2.1 (Polynomial IOP)**.** A *polynomial interactive oracle proof* for a relation $\mathcal{R}$ is a public-coin interactive protocol where in each round:

- The prover sends one or more *polynomial oracles* $p_i \in \mathbb{F}[X]$ (each of bounded degree).
- The verifier sends a random challenge $c \in \mathbb{F}$.

At the end of the protocol, the verifier may query each oracle at a small number of points, learning the polynomial's evaluations at those points. The verifier's final accept/reject is a deterministic function of the challenges, evaluations, and the public input x.

We define completeness, soundness, and knowledge soundness as usual.

A PIOP differs from a vanilla interactive argument in one essential respect: the verifier does not read the prover's messages in full — it queries only a few evaluations of each polynomial oracle. This is what renders the verifier "succinct" even when the polynomials themselves have large degree.

2.2 Compiling a PIOP into a SNARK

Construction 2.2 (PIOP-to-SNARK compiler)**.** Given a PIOP Π for $\mathcal{R}$, a PCS scheme $(\mathsf{Setup}, \mathsf{Commit}, \mathsf{Open}, \mathsf{Verify})$ for $\mathbb{F}[X]$, and a random oracle H, the compiler produces the SNARK:

1. Setup: run the PCS setup to obtain pp.
2. Prover: simulate Π, replacing each polynomial oracle p_i by its PCS commitment C_i, and each verifier-challenge / oracle-query round by the corresponding PCS opening. Use Fiat–Shamir (the *Crypto Guide*) to derive challenges from the transcript.

3. Verifier: parse the transcript, recompute Fiat–Shamir challenges, verify each PCS opening, and run the PIOP's final accept/reject check.

Theorem 2.3 (Soundness preservation)**.** *If* Π *has round-by-round (equivalently, state-restoration) knowledge soundness* κ_Π *and the PCS is extractable with extraction error* κ_{PCS} *(the PCS definition of the* Crypto Guide*), the compiled SNARK has knowledge soundness* $O(Q \cdot (\kappa_\Pi + \kappa_{\mathrm{PCS}}))$ *in the random oracle model, where* Q *is the adversary's RO-query budget.*

The query budget multiplies κ_Π as well as κ_{PCS}: step 2 derives the PIOP challenges by Fiat–Shamir, so a cheating prover that re-randomises a commitment redraws every subsequent challenge, obtaining up to Q independent draws of each — the grinding attack. Round-by-round soundness confines this loss to the single factor Q; for an r-round PIOP with plain knowledge soundness only, the naive Fiat–Shamir analysis loses a factor $Q^{O(r)}$ (§3).

This factoring is of considerable utility: research on PIOPs (PLONK, Marlin, Spartan, HyperPlonk, ...) proceeds independently of research on polynomial commitment schemes, and any compatible pair combines into a new SNARK that inherits the PIOP's relation and the commitment scheme's cryptographic profile. This document develops the one PCS that Halo 2 uses — the inner-product argument (§5.5) — and treats the PIOP/PCS split only insofar as it clarifies the Halo 2 construction.

2.3 A worked example: a PIOP for univariate evaluation

To render the abstraction concrete, consider a minimal PIOP for the simplest non-trivial relation: "the prover knows a polynomial p of degree $\leq d$ such that $p(z) = y$ for a public (z, y)."

Construction 2.4 (PIOP for univariate evaluation)**.** On input (z, y) and prover witness p:

1. Prover sends the polynomial oracle p and the quotient oracle $q := (p - y)/(X - z)$.
2. Verifier samples a random challenge $r \in \mathbb{F}$.
3. Verifier queries p and q at r.
4. Verifier accepts iff $p(r) - y = q(r) \cdot (r - z)$.

Completeness is immediate (q is a polynomial since $p(z) - y = 0$). Soundness follows from Schwartz–Zippel: if $(X - z) \nmid (p(X) - y)$, then $p(X) - y - q(X)(X - z)$ is a non-zero polynomial of degree $\leq d$, and its random evaluation is zero with probability $\leq d/|\mathbb{F}|$.

Compiling this PIOP with the IPA-PCS, applying Fiat–Shamir, yields a non-interactive SNARK for the same relation. The PCS handles polynomial hiding and binding; the PIOP handles the relation logic. This separation is precisely the modular SNARK design pattern.

2.4 PLONK as a PIOP

PLONK (§5.1) is a more elaborate PIOP. Its oracles are:

- One advice polynomial per advice column.
- A permutation running-product polynomial Z_{perm}.
- Two permuted lookup columns A', S' per lookup.
- One lookup running-product polynomial per lookup.
- A quotient polynomial h (split into chunks).

Its challenges are, in order: θ (lookup column compression), β, γ (the permutation and lookup products), y (the gate-equation combination), and x (the final evaluation point). Its final check is the PLONK gate equation $C(x) = h(x) \cdot Z_H(x)$, which the verifier checks from the evaluations of the oracles at x (and possibly ωx).

This is precisely the PIOP that phases 1–5 of the Halo 2 protocol of §5.7 compile, phase 5's claimed evaluations answering its oracle queries; phases 6–7 bind those answers to the commitments — the multipoint reduction (the challenges $x_1, \ldots, x_4$ and the combined polynomial f^*) followed by a single IPA-PCS opening of f^*.

3 A rigorous treatment of knowledge soundness

We now develop knowledge soundness with the care needed for arguments about modern SNARKs. The treatment combines the rewinding-extractor framework (the *Crypto Guide*) with the algebraic group model.

3.1 Definitional refinements

The bare definition of the *Crypto Guide* suffices for Σ-protocols but is awkward for compound systems. We record the following standard refinement.

Remark 3.1 (Witness-extended emulation)**.** An interactive argument $(\mathcal{P}, \mathcal{V})$ has *witness-extended emulation* if there is a PPT emulator $\mathcal{E}$ such that for every PPT prover $\mathcal{P}^*$ and every input x:

- Run $\mathcal{P}^*$ honestly with $\mathcal{V}$; let τ be the transcript and b the verifier's accept/reject bit.
- Run $\mathcal{E}^{\mathcal{P}^*}(x)$; it outputs (τ', b', w).

The distributions of (τ, b) and (τ', b') are computationally indistinguishable, and moreover whenever $b' = 1$, $(x, w) \in \mathcal{R}$ except with negligible probability.

WEE captures simultaneously two properties: "the extractor produces a witness whenever the protocol accepts" and "the extractor's external behaviour resembles a normal protocol execution." It is the appropriate notion for arguments embedded in larger protocols, such as the batched verification of many proofs at once (§5.6); plain knowledge soundness, however, suffices for this volume's statements.

3.2 The algebraic group model

The IPA's knowledge-soundness proof proceeds through the tree extractor of §5.5, with knowledge error $O(d/|\mathbb{F}|)$. The interactive tree extractor costs $3^{\log_2 d} = d^{\log_2 3}$ transcripts and runs in expected polynomial time (§3.3); the looseness arises for the non-interactive argument, where the naive Fiat–Shamir analysis of the $\log d$-round protocol loses a factor $q^{O(\log d)}$ in the adversary's random-oracle query count q — quasi-polynomial for polynomial q. The *algebraic group model* (AGM; Fuchsbauer, Kiltz, Loss; CRYPTO 2018) removes this loss by restricting the adversary's group operations.

Definition 3.2 (Algebraic adversary)**.** An *algebraic adversary* against a group $\mathbb{G}$ is a PPT algorithm that, whenever it outputs a group element $P \in \mathbb{G}$, additionally outputs an explicit linear combination $P = \sum_i [a_i] Q_i$ expressing P as a combination of previously-seen group elements Q_i.

The AGM sits between the standard model (no restriction on the adversary) and the generic group model (group operations available only through an oracle). Most natural adversaries are algebraic — they compute group elements by known scalar multiplications and additions — so the heuristic is regarded as conservative. In the AGM the forking loss disappears: an algebraic

prover outputs every group element together with an explicit linear combination of the elements it has seen, so the extractor reads candidate witnesses directly off these representations instead of reassembling them from forked transcripts. Ghoshal and Tessaro (CRYPTO 2021) carry out this analysis for Bulletproofs-style inner-product arguments, proving tight state-restoration soundness — and hence tight Fiat–Shamir security — in the AGM. The accumulation analysis of BCMS20 (Bünz, Chiesa, Mishra, Spooner; TCC 2020), by contrast, does not invoke the AGM: it proves security in the random oracle model under the DL assumption (Theorem 5.7).

3.3 Tree-extractor analysis

To render the tree extractor concrete, consider the key step at each round of the IPA halving (see §5.5):

Lemma 3.3 (Single-round extraction)**.** *In a single round of the IPA at depth j, given three accepting sub-transcripts that share the round's first message (L_j, R_j) but have three challenges u_j with pairwise distinct squares ($u_j \neq \pm u'_j$), the extractor recovers the round-j witness halves $\vec{a}_{\mathrm{lo}}^{(j)}, \vec{a}_{\mathrm{hi}}^{(j)}$ together with the consistency of the L_j, R_j cross-terms via 3×3 Vandermonde inversion (rows $(u_j^{-2}, 1, u_j^2)$), since the folded relation is quadratic in u_j.*

Sketch. The three accepting transcripts give three accepting verifier equations. Because the folded relation is quadratic in u_j (the three monomials $u_j^{-2}, 1, u_j^2$), treating these as three linear equations in three unknowns (the cross-terms and the folded commitment), the system has a unique solution because the 3×3 Vandermonde determinant with rows $(u_j^{-2}, 1, u_j^2)$ is non-zero exactly when the squares u_j^2 are pairwise distinct ($u_j \neq \pm u'_j$). Excluding the bad challenge collisions contributes an additive $O(1/|\mathbb{F}|)$ term per round to the knowledge error. □

The full extraction requires three accepting transcripts at each of the $k = \log_2 d$ rounds, giving a tree of $3^k = d^{\log_2 3} \approx d^{1.585}$ leaves — polynomial in d, though more than the $O(d)$ a binary tree would cost. In the AGM, each prover invocation is direct (the extractor reads the algebraic representation); in the standard model with forking, each invocation costs $1/\varepsilon$ expected queries to the prover, where ε is the cheating prover's acceptance probability, yielding total cost $O(d^{\log_2 3} \cdot \mathsf{poly}(\lambda)/\varepsilon)$.

3.4 Concrete versus asymptotic security

The literature on knowledge soundness contains numerous tightness gaps. The more important ones include:

- Bulletproofs' naive Fiat–Shamir knowledge-soundness analysis loses a factor $q^{O(\log d)}$, exponential in the round count $\log d$ with base the query count q; the interactive standard-model extraction itself is polynomial ($d^{\log_2 3}$ transcripts, expected polynomial time).
- Halo 2's deployed parameters assume the AGM and/or the random oracle model heuristically.

This reflects the practical reality of deployed SNARKs: tight reductions in idealised models, loose reductions in standard models, and deployment selecting the former.

4 Recursive proof composition and accumulation schemes

We now turn to recursive proof composition: proving, inside one proof, that another proof verifies. This is what makes Halo 2's combination of "no trusted setup" with recursion possible. Orchard's mainnet deployment does not recurse, but the accumulation structure shapes the deployed verifier nonetheless — the deferred MSM and the batched verification of §5.6 — so we develop it with care.

4.1 Recursion and its two classical obstacles

Consider a computation iterated many times, $z_n = f^{(n)}(z_0) := f(f(\dots f(z_0)\dots))$ — Valiant's *incrementally-verifiable computation* (TCC 2008) — or, more generally, a DAG of computations each consuming the proofs of its predecessors — *proof-carrying data* (Chiesa, Tromer; ICS 2010). The natural construction is *recursive SNARK composition*: at step $i+1$ the prover proves "$z_{i+1} = f(z_i)$, *and* there exists a proof π_i that the verifier circuit for step i accepts." One proof then attests to the entire history, and the per-step proving cost is independent of the history's length.

For this to work, the SNARK's verifier must itself be efficiently expressible as a circuit, and here the classical instantiations meet two obstacles:

1. Pairing-based SNARKs have constant-size, constant-time verifiers, but the verifier evaluates a pairing — costly to express algebraically inside a circuit, since it requires extension-field arithmetic — and these SNARKs need a trusted setup besides.

2. Transparent SNARKs built on the IPA need no pairing and no trusted setup, but their verifiers run in linear time — the $O(d)$ MSM of §5.5.2. A linear-time check dominates the verifier circuit and negates the per-step efficiency.

Halo (Bowe–Grigg–Hopwood 2019) defeats obstacle 2 directly — deferring the linear-time part of verification instead of recomputing it at every step — and sidesteps obstacle 1 by using the IPA, which involves no pairing. BCMS20 formalised the deferral as an *accumulation scheme*.

4.2 Accumulation schemes

Definition 4.1 (Accumulation scheme, informal)**.** For a predicate Φ (for example, "this IPA opening is correct"), an *accumulation scheme* consists of:

- An *accumulator*, a compact data structure representing a batch of claims about Φ.
- A *folding* algorithm: given an old accumulator and a new claim about Φ, it produces a new accumulator that represents both — the new accumulator is valid only if the old one was valid and the new claim holds. Folding is fast: $O(\log d)$ in the IPA instantiation below.
- A *decider*, which is expensive (linear-time) but which the scheme invokes only once at the very end to validate the accumulator. If the decider accepts, every claim folded in so far is true.

For the IPA-PCS such a scheme exists, and it resolves the recursion problem in the IPA setting: the in-circuit verifier checks only the $O(\log d)$ "succinct" part of an IPA proof and folds the $O(d)$ "deferred" MSM check into a running accumulator. The decider pays the deferred work once, at the end of the entire chain. We state the scheme precisely as Theorem 5.7, once the IPA construction of §5.5 is in hand.

4.3 The necessity of a cycle of curves

The accumulation scheme's in-circuit folding involves arithmetic on group elements of the underlying curve. For this to be efficient inside the circuit, the group's base field must equal the circuit's native field. Since the circuit's native field is the scalar field of the curve on which the prover commits the SNARK — which in turn is generally different from the curve's own base field — a difficulty arises: an E_p-IPA proof's group elements have coordinates in $\mathbb{F}_p$, so verifying it natively would require a circuit whose native field is $\mathbb{F}_p$. But a SNARK's verifier circuit naturally operates over the curve's *scalar* field $\mathbb{F}_q$ (where $q = |E_p|$): the Fiat–Shamir challenges and folding scalars are $\mathbb{F}_q$ elements. Curves with $p = q$ do exist — the *anomalous* curves, with trace of Frobenius 1 — but on them the discrete logarithm is computable in polynomial time (Smart; Semaev; Satoh–Araki, 1998–99), so no *secure* curve has $\mathbb{F}_p = \mathbb{F}_q$, and no one curve E_p can host both the proof's coordinates and its own verifier circuit natively.

The resolution is a *2-cycle* of elliptic curves, where $|E_p| = q$ and $|E_q| = p$. Then:

- A circuit committed on E_q verifies a proof generated over E_p; that circuit's native field is the scalar field of E_q, namely $\mathbb{F}_p$ (since $|E_q| = p$) — which coincides with the coordinate field of E_p, enabling native manipulation of E_p's group elements.
- A circuit over $\mathbb{F}_q$ verifies the next proof, generated over E_q.
- Proofs alternate between the two curves.

This is precisely the Pasta cycle of the *Math Guide*, where the two curves are Pallas (over $\mathbb{F}_{p_{\mathsf{Pallas}}}$, order p_{Vesta}) and Vesta (over $\mathbb{F}_{p_{\mathsf{Vesta}}}$, order p_{Pallas}).

5 The Halo 2 proof system

We now assemble the abstractions of the *Crypto Guide*–§4 into the concrete proof system Halo 2: a transparent PLONKish SNARK whose polynomial commitment is the inner-product argument over the Pasta cycle. This section develops Halo 2 to the level of detail required to state *what an Orchard Action proof actually is*, the subject of the *Ironwood Guide*.

5.1 PLONKish arithmetisation

Definition 5.1 (PLONKish constraint system)**.** Fix a prime field $\mathbb{F}$ and an integer k defining a multiplicative subgroup $H := \{\omega^0, \dots, \omega^{n-1}\} \subset \mathbb{F}^*$ of size $n := 2^k$, where ω is a primitive n-th root of unity. A *PLONKish circuit* consists of:

- Columns partitioned into *fixed* (preprocessed), *advice* (prover-witness), and *instance* (public-input).
- A maximum constraint degree d.
- A finite sequence of *custom gates* $g_t(\vec{x}_0, \vec{x}_1, \dots) = 0$ — multivariate polynomials of degree $\le d$ in column values at the current row and at offsets $\omega^r X$ for fixed small r.
- A subset $S \subseteq H \times [\#\text{cols}]$ of *equality-constraint cells* controlled by a permutation σ.
- A set of *lookup arguments*: input tuples $(A_0, \dots, A_{m-1})$ and table tuples $(S_0, \dots, S_{m-1})$, with the assertion that for every row $i \in [0, n)$, $(A_0(\omega^i), \dots, A_{m-1}(\omega^i))$ matches some row of the table.

Vanilla PLONK (Gabizon–Williamson–Ciobotaru, 2019) employs a fixed five-selector universal gate

$$q_M(X)a(X)b(X) + q_L(X)a(X) + q_R(X)b(X) + q_O(X)c(X) + q_C(X) \equiv 0 \pmod{Z_H(X)},$$

where $Z_H(X) := X^n - 1$ is the *vanishing polynomial* of the domain H (it is zero exactly on H, so "$\equiv 0 \bmod Z_H$" means "zero at every row"). "PLONKish" generalises this by permitting designer-chosen gates of higher arity and degree, each gated by a selector $q_i(X)$ vanishing on rows where the gate is inactive. The Orchard Action circuit uses ten advice columns plus a small fixed-column lookup table for Sinsemilla; its custom gates encode the algebraic constraints of the Action statement (the *Ironwood Guide*) together with the complete- and incomplete-addition arithmetic for in-circuit elliptic-curve operations (the *Math Guide* names the distinction).

Polynomial representation. The Lagrange basis $\ell_j(X)$ for H, where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ for $j' \neq j$, interpolates each column into a polynomial of degree $< n$ over H. The prover commits to each polynomial via the IPA-PCS (§5.5). All subsequent arguments are *polynomial identities* on these committed polynomials, verifiable by querying them at random points.

5.2 From statement to circuit: arithmetisation in practice

Definition 5.1 states what a PLONKish circuit *is*; it does not say how a statement *becomes* one. This subsection traces that passage through the deployed code: a real custom gate (the ECC chip's incomplete point addition), two real lookup declarations (the ten-bit range check and the Sinsemilla generator table), and the synthesis machinery that turns witness data into the column vectors on which every argument of this section operates. Our sources are `halo2_proofs` 0.3.2 and `halo2_gadgets` 0.4.0, as resolved by `orchard` 0.13.1, the version we cite, with file paths relative to each crate's `src/`. The deployed `orchard` (0.15.0) keeps `halo2_proofs` 0.3.2 but resolves `halo2_gadgets` 0.5.0, the release carrying the NU6.2 circuit correction (ZIP 257; the *Ironwood Guide*); every gate and lookup declaration cited below is identical in the two releases.

Shape first, data second. A circuit, to `halo2_proofs`, is a type implementing the `Circuit` trait, whose two principal methods split the construction into phases (`halo2_proofs`, `plonk/circuit.rs`). The static method `configure` receives a mutable `ConstraintSystem` and declares the *shape*: it mints fixed, advice, and instance columns, allocates selectors, registers gates and lookups, and marks columns as equality-enabled. No witness exists at this point; `configure` is not even handed an instance of the circuit type. The method `synthesize` runs later, with the data, and fills cells. The split is the proof system's division of labour: everything `configure` declares is public and determines the verifying key — the verifier's entire view of the circuit — while everything `synthesize` writes into advice cells is the prover's witness. (Key generation also runs `synthesize` once, against a witness-free backend that records fixed-cell values, selector activations, and copy constraints while ignoring advice; the prover runs the same code against a backend that records only advice. One synthesis procedure, two observers: `Assembly` in `plonk/keygen.rs` and `WitnessCollection` in `plonk/prover.rs`.)

Gates are queried identities. A call `meta.create_gate(name, closure)` defines one custom gate. Inside the closure, each `query_advice(column, Rotation(r))` — and its fixed and instance analogues — yields a symbolic handle to "the value of this column r rows below the gate's own row": the offsets $\omega^r X$ of Definition 5.1, realised at evaluation time by querying the column polynomial at $\omega^r x$ (`halo2_proofs`, `poly/domain.rs`, `rotate_omega`). The closure combines the handles by field arithmetic into an expression tree (`Expression`: constants, selector nodes, queried cells, negations, sums, products, scalings), one tree per constraint, and the helper `Constraints::with_selector` multiplies every constraint c by the gate's selector expression, storing $q \cdot c$ (`plonk/circuit.rs`). This helper is the mechanical origin of the "selector times polynomial" shape of every deployed gate but one — the generalisation, gate by gate, of the fixed vanilla-PLONK equation of §5.1. (The exception, the ECC chip's witness-point gate, multiplies by hand: its $(q \cdot x) \cdot (y^2 - x^3 - 5)$ and the helper's $q \cdot \big(x \cdot (y^2 - x^3 - 5)\big)$ are equal polynomials but different expression trees, and the pinned verifying key freezes the tree; `halo2_gadgets`, `ecc/chip/witness_point.rs`.) A selector is virtual at this stage: a named boolean column, off everywhere by default, enabled row by row during synthesis. At key generation, `compress_selectors` packs the selectors into genuine fixed columns — several to a column wherever the circuit's degree bound leaves headroom — and rewrites every selector node into a fixed-column expression (`plonk/circuit.rs`, `plonk/keygen.rs`). The deployed Action circuit declares 56 selectors, whose compressed columns are among the 29 fixed columns pinned in its verifying key; the pinned count is the post-compression one (`orchard` 0.13.1, `src/circuit_description`; since 0.14 the file lives at `src/circuit_data/circuit_description_insecure`, and the corrected post-NU6.2 key beside it, `circuit_description_fixed`, pins the identical constraint system, differing only in its permutation commitments).

Regions, the layouter, and wiring. Data enters at synthesis through a `Layouter`. A chip requests a *region* — a small rectangular window of cells addressed by relative offsets — fills it cell by cell, and leaves the choice of the region's absolute starting row to the *floor planner* (`halo2_proofs`, `circuit.rs`). Values flow between regions not by position but by *copy constraints*: the call `enable_equality(column)` enrols a column in the single global permutation of §5.3, and `constrain_equal` — or the assign-and-constrain combination `copy_advice` — merges the two cells' cycles in that permutation (`plonk/permutation/keygen.rs`; the halo2 book's *Permutation argument* chapter documents the cycle-merging). Public inputs bind the same way, an advice cell copy-constrained to an instance cell (§5.8). The deployed circuit uses the measuring `V1` floor planner, which records every region's shape in a witness-free pass, sorts the regions by advice area, and first-fits the largest ones first (`circuit/floor_planner/v1.rs`; the sort and first-fit in `v1/strategy.rs`); the `orchard` crate additionally pins the planner's sort to a vendored Rust 1.56.1 `pdqsort` (feature `floor-planner-v1-legacy-pdqsort`) so that the layout — and with it the consensus verifying key — is identical across platforms and toolchains.

A deployed gate: incomplete point addition. The ECC chip's incomplete-addition gate (`halo2_gadgets`, `ecc/chip/add_incomplete.rs`) computes $P + Q = R$ on Pallas under the promises $P, Q \neq \mathcal{O}$ and $x_p \neq x_q$. Its configuration is one selector $q_{\text{add-incomplete}}$ and four equality-enabled advice columns `x_p`, `y_p`, `x_qr`, `y_qr`; the gate queries x_p, y_p, x_q, y_q at the current row and x_r, y_r at the next row of the *same* two columns `x_qr`, `y_qr` (hence their names: Q on the selector row, R below it). The two constraints, named `x_r` and `y_r` in the source, are

$$q_{\text{add-incomplete}} \cdot \left((x_r + x_q + x_p)\,(x_p - x_q)^2 - (y_p - y_q)^2 \right) = 0, \tag{1}$$

$$q_{\text{add-incomplete}} \cdot \left((y_r + y_q)\,(x_p - x_q) - (y_p - y_q)\,(x_q - x_r) \right) = 0. \tag{2}$$

These are the affine chord formulas (the *Math Guide*) with the slope eliminated: for $x_p \neq x_q$ they are equivalent to $x_r + x_q + x_p = \lambda^2$ and $y_r + y_q = \lambda\,(x_q - x_r)$ with $\lambda := (y_p - y_q)/(x_p - x_q)$, multiplied through by $(x_p - x_q)^2$ and $(x_p - x_q)$ respectively to clear the denominators (the halo2 book's *Addition* chapter records the same clearing). The raw degrees are three and two; with the selector factor, four and three — well inside the deployed circuit's constraint-degree bound of nine (below). The *complete*-addition companion gate, which branches over the identity, doubling, and inverse cases that this gate excludes, needs twelve constraint polynomials of degree up to six and five auxiliary witness cells per invocation (`ecc/chip/add.rs`); incompleteness is a bargain wherever its promises can be kept.

Synthesis for one invocation (`assign_region`, same file) makes the witness path concrete. The chip enables the selector at the region's row i; copies the four input coordinates into that row with `copy_advice`, which both assigns each cell and copy-constrains it back to its source; computes, *out of circuit*,

$$\lambda = \frac{y_q - y_p}{x_q - x_p}, \qquad x_r = \lambda^2 - x_p - x_q, \qquad y_r = \lambda\,(x_p - x_r) - y_p;$$

and assigns x_r, y_r into row $i+1$ (Table 1). The slope λ is never written to any cell: the prover uses it to compute the answer, and the committed identities (1)–(2) verify the answer without it. The division is not even performed here — coordinates are carried as deferred fractions (`halo2_proofs`, `plonk/assigned.rs`), and every denominator falls to a single batch inversion when synthesis ends (`plonk/prover.rs`, `batch_invert_assigned`). The output cells are returned as a new non-identity point with no further on-curve check — sound because, once the call site guarantees distinct x-coordinates on non-identity inputs, the two identities force $R = P + Q$, a non-identity curve point.

Incompleteness, and where it is discharged. Reading the two polynomials at the excluded inputs shows exactly what "incomplete" costs. If $Q = -P \neq \mathcal{O}$ (so $x_p = x_q$ and $y_p = -y_q \neq 0$),

row	x_p	y_p	x_qr	y_qr	$q_{\text{add-incomplete}}$
i	x_p	y_p	x_q	y_q	1
$i+1$	–	–	x_r	y_r	0

Table 1: The incomplete-addition region: two rows by four advice columns (`halo2_gadgets`, `ecc/chip/add_incomplete.rs`). The four input cells x_p, y_p, x_q, y_q on the selector row i are copy-constrained to the cells they came from; the result cells x_r, y_r occupy the `x_qr`, `y_qr` columns of row $i+1$, where the gate's `Rotation::next()` queries find them. Dashes mark cells the gate does not use.

constraint (1) collapses to $-(2y_p)^2 \neq 0$: no assignment satisfies the gate — a completeness failure, never a soundness one. If $Q = P$, both polynomials vanish identically and (x_r, y_r) is *unconstrained*: a reachable doubling would let the prover claim any result. If an input were the identity in the gadget's $(0, 0)$ affine encoding, the constraints would accept a garbage output. The deployed circuit discharges the three exclusions in three different places. Non-identity travels with the type: the gate's inputs are `NonIdentityEccPoint` values. A fresh point entering the chip is witnessed under the gate

$$q_{\text{point-non-id}} \cdot \left(y^2 - x^3 - 5\right) = 0 \qquad (\texttt{halo2\_gadgets}, \texttt{ecc/chip/witness\_point.rs}),$$

which $(0, 0)$ cannot satisfy because 5 is not a square and -5 is not a cube in $\mathbb{F}_{p_{\text{Pallas}}}$ (the halo2 book's *Addition* chapter) — but no fresh witness ever reaches incomplete addition. In the deployed circuit the incomplete-addition gate fires only inside fixed-base scalar multiplication (`ecc/chip/mul_fixed.rs`), which sums precomputed windowed multiples $[(k + 2) \cdot 8^w]B$ of a fixed base B. There the addends are assembled by the type's unchecked constructor (`from_coordinates_unchecked`) and pinned instead to the precomputed window table by `mul_fixed`'s own coordinate gates (`coords_check`, same file): the x-coordinate must equal a degree-seven polynomial in the window value whose fixed coefficients interpolate the window's eight table x-coordinates, and the y-coordinate must satisfy $y + z_w = u^2$ against a per-window fixed z_w and a witnessed u, alongside an on-curve check — together pinning the point to the table entry selected by the window digit, and no entry of the table is the identity. The running accumulator is a sum of such entries under this very gate; the type carries the non-identity guarantee from either construction site to the gate, which re-checks nothing. The distinctness $x_p \neq x_q$ is enforced by no constraint at all; it too is structural: the window tables store $[(k + 2) \cdot 8^w]B$ rather than the naive $[(k + 1) \cdot 8^w]B$ precisely because the naive offset admits a window collision that produces the fatal doubling (the halo2 book's *Fixed-base scalar multiplication* chapter). Prover-side, `assign_region` rejects a *known* bad witness with an error — a developer diagnostic, not a constraint. (The circuit's other incomplete additions — the variable-base ladder's fused double-and-add rounds and Sinsemilla's two per chunk, the ones the *Ironwood Guide*'s gadget survey counts — are separate gates built on the same discipline; `ecc/chip/mul/incomplete.rs`, `sinsemilla/chip.rs`.)

Non-algebraic relations become lookups. A gate can assert only polynomial identities. "This value has ten bits" and "this point is the j-th Sinsemilla generator" are not identities but set memberships, and the circuit declares them as lookups: a call `meta.lookup(closure)` returns pairs of an input *expression* and a table column, and pushes one lookup argument onto the constraint system (`halo2_proofs`, `plonk/circuit.rs`). Each declaration is one instance of the argument of §5.4, the input expressions playing $A_0, \ldots, A_{m-1}$ and the table columns $S_0, \ldots, S_{m-1}$. The deployed circuit carries exactly three (`orchard` 0.13.1, `src/circuit_description`): one shared range-check declaration, and the Sinsemilla generator declaration twice — once per Sinsemilla chip instance, each on its own half of the ten advice columns.

All three consume the same table. Loaded once at synthesis, it holds, for $0 \le j < 2^{10}$,

$$\text{row } j: \quad \big(j,\ X(S[j]),\ Y(S[j])\big), \qquad S[j] := \mathsf{GroupHash}\big(\texttt{"z.cash:SinsemillaS"},\ \mathrm{LE}_{32}(j)\big),$$

in three table columns $(t_{\mathrm{idx}}, t_x, t_y)$; the generators ship as precomputed constants that the crate's own tests re-derive from the hash (`sinsemilla` crate, `src/sinsemilla_s.rs`; `halo2_gadgets`, `utilities/lookup_range_check.rs` for the loading). After the assignment, the layouter fills every remaining usable row of each table column with that column's row-0 value (`halo2_proofs`, `circuit/table_layouter.rs`); a defaulted input row therefore still matches a genuine table row, and the type `TableColumn` hides its underlying fixed column exactly so that no chip can bypass this default-filling.

The range check is one declaration serving two modes (`halo2_gadgets`, `utilities/lookup_range_check.rs`):

$$q_{\mathrm{lookup}} \cdot \Big(q_{\mathrm{running}} \cdot \big(z_i - 2^{10}\, z_{i+1}\big) + (1 - q_{\mathrm{running}}) \cdot z_i\Big) \ \longmapsto\ t_{\mathrm{idx}}, \tag{3}$$

with z_i and z_{i+1} the running-sum advice column at the current and next rotations. With $q_{\mathrm{running}} = 1$ the input is a running-sum step: the prover witnesses $z_{i+1} = (z_i - a_i) \cdot 2^{-10}$ on successive rows, the expression recovers the limb $a_i = z_i - 2^{10} z_{i+1}$, and membership in $t_{\mathrm{idx}} = \{0, \ldots, 2^{10} - 1\}$ asserts that each limb of the decomposition has ten bits; a *strict* decomposition additionally copy-constrains the final z to the constant 0, range-constraining the decomposed element to $10w$ bits for w limbs. With $q_{\mathrm{running}} = 0$ the cell itself is looked up (checks of fewer than ten bits add a small bit-shift gate in the same file). Every range check in the Action circuit funnels through this one declaration, on a single advice column.

The Sinsemilla declaration is the three-column instance (`halo2_gadgets`, `sinsemilla/chip/generator_table.rs`):

$$\Big(\ q_{S1} \cdot m,\ \ q_{S1} \cdot x_p + (1 - q_{S1}) \cdot X(S[0]),\ \ q_{S1} \cdot y_p + (1 - q_{S1}) \cdot Y(S[0])\ \Big) \ \longmapsto\ (t_{\mathrm{idx}},\, t_x,\, t_y), \tag{4}$$

where $m = z_i - 2^{10} \cdot q_{\mathrm{run}} \cdot z_{i+1}$ recovers the ten-bit message word from the hash's own running-sum decomposition (with $q_{\mathrm{run}} := q_{S2} - q_{S2}(q_{S2} - 1)$, an expression in a fixed column $q_{S2} \in \{0, 1, 2\}$ that evaluates to 1 on the interior rows of a message piece and to 0 on final rows, where the trailing running sum is implicitly zero), and where y_p is not a witnessed cell at all but the derived expression $Y_A \cdot 2^{-1} - \lambda_1(x_A - x_p)$ over the chip's double-and-add columns (`ecc/chip/mul/incomplete.rs` supplies $Y_A := (\lambda_1 + \lambda_2)(x_A - x_R)$ and $x_R := \lambda_1^2 - x_A - x_p$). Lookup inputs, in other words, may be arbitrary expressions over queried cells, not merely cells; the chip proves that the generator it is adding *is* $S[m]$ without ever spending an advice cell on $y_{S[m]}$. The $(1 - q_{S1})$ branches default every row on which the chip is inactive to table row 0, so the membership assertion of §5.4 holds on all rows. One API restriction is visible here: the selectors gating lookup inputs (q_{lookup}, q_{running}, q_{S1}) must be declared as `complex_selectors` — simple selectors are reserved for gates, where the key-generation compression is licensed to rewrite them (`halo2_proofs`, `plonk/circuit.rs`).

The handoff. When `synthesize` returns, the construction has produced nothing exotic: every column — advice, fixed, instance — is a vector of $n = 2^k$ field elements; in the advice columns the deferred fractions are resolved by one batch inversion and the reserved trailing rows filled with blinding randomness (§5.9, Remark 5.2). The prover commits to each advice column directly in the Lagrange basis (`halo2_proofs`, `poly/commitment.rs`, `commit_lagrange` — the same group element as committing the interpolated coefficients, so the commitment even precedes interpolation) and then interpolates each column over H by an inverse FFT (`poly/domain.rs`, `lagrange_to_coeff`): these are precisely the column polynomials of §5.1. Each gate's stored expression tree is then evaluated with the committed column polynomials substituted for the queried cells — a query at rotation r reading the polynomial at $\omega^r X$ — and the results are folded with powers of the challenge y and divided by $Z_H(X) = X^n - 1$ (`plonk/prover.rs`;

`plonk/vanishing/prover.rs`; `divide_by_vanishing_poly` in `poly/domain.rs`). The gate identities declared in `configure` have become the quotient $h(X)$ of Phase 4 (§5.7). This is the point where the toy of *How Halo 2 Proves* resumes — its § *From the grid to polynomials* interpolates a four-row trace and assembles exactly this $G(X)$ whose divisibility by Z_H the rest of the protocol certifies — and where the *Ironwood Guide*'s gadget stack bottoms out: each gadget of its survey is a bundle of `configure`-time gates and lookups plus `synthesize`-time regions, and after synthesis nothing remains of the stack but rows in these same columns. The scale-up from the toy is numerical, not conceptual. The deployed Action circuit has $k = 11$ ($n = 2048$ rows), the ten advice columns of §5.1, one instance column, and twenty-nine fixed columns, with a constraint-degree bound of nine across its gates and permutation and lookup expressions — so the quotient splits into $9 - 1 = 8$ chunks, and the prover's FFTs run over an extended domain of size $2^{14} = 8 \cdot 2^{11}$ (`orchard` 0.13.1, `src/circuit_description`; `halo2_proofs`, `poly/domain.rs`). The next two subsections supply the arguments this construction leans on: the permutation argument behind every copy constraint, and the lookup argument behind declarations (3) and (4).

5.3 The permutation (equality) argument

A PLONK-style permutation argument, which Halo 2 generalises to multiple columns, enforces equality constraints between distinct cells. Each cell (i, j) (column i, row j) receives a unique label $\delta_i \cdot \omega^j$ with δ_i chosen so all products are distinct as (i, j) ranges over the trace. The prover writes σ-permuted polynomials $s_i(X)$ with $s_i(\omega^j) = \delta_{i'} \cdot \omega^{j'}$ when $\sigma(i, j) = (i', j')$.

Given Fiat–Shamir challenges $\beta, \gamma \in \mathbb{F}$, the prover commits to a running-product polynomial $Z_{\text{perm}}(X)$ asserting, where $v_i(X)$ denotes the Lagrange interpolation (§5.1) of the i-th column participating in the permutation,

$$\prod_{i,j} \frac{v_i(\omega^j) + \beta\, \delta_i\, \omega^j + \gamma}{v_i(\omega^j) + \beta\, s_i(\omega^j) + \gamma} = 1.$$

The assignment satisfies every equality constraint (cells related by σ carry equal values) precisely when the denominator factors are a permutation of the numerator factors, in which case the product equals 1 for every β, γ; conversely, if some constraint is violated, the product at the sampled β, γ equals 1 with probability at most $O(N/|\mathbb{F}|)$ by Schwartz–Zippel, where N is the number of cells participating in the permutation. The verifier checks this by opening Z_{perm} at x and ωx, verifying the boundary $Z_{\text{perm}}(\omega^0) = 1$ and the row-by-row update. The per-proof verifier work is $O(m + \log n)$, where m denotes the number of columns participating in the permutation.

5.4 The lookup argument

Halo 2's lookup argument is structurally simpler than plookup (Gabizon–Williamson 2020) and supports arbitrary (possibly non-fixed) tables.

Objective. Establish that every entry of column $A(X)$ on the active domain appears as some entry of column $S(X)$.

Construction. The prover supplies polynomials $A'(X), S'(X)$ where (i) A' is a permutation of A with like values grouped into contiguous runs, and (ii) S' is a permutation of S in which the first row of each run of like values in A' has the matching value in S'. The prover commits to A' and S'; after receiving challenges β, γ, it commits to a running-product column $Z_{\text{lookup}}(X)$ and

proves:

$$\begin{aligned}
\ell_0(X)\big(1 - Z_{\text{lookup}}(X)\big) &= 0,\\
Z_{\text{lookup}}(\omega X)(A'(X)+\beta)(S'(X)+\gamma) - Z_{\text{lookup}}(X)(A(X)+\beta)(S(X)+\gamma) &= 0,\\
\ell_0(X)(A'(X) - S'(X)) &= 0,\\
(A'(X) - S'(X))(A'(X) - A'(\omega^{-1}X)) &= 0.
\end{aligned}$$

The first two constraints, with the boundary conditions of Remark 5.2, force A' and S' to be permutations of A and S respectively; the third aligns row 0 ($A'_0 = S'_0$), where the wrap-around reference $A'(\omega^{-1}X)$ deprives the fourth constraint of force. The fourth constrains every subsequent row either to start a new run aligned with S' ($A'_i = S'_i$) or to continue the previous one ($A'_i = A'_{i-1}$). Inductively, every value of A equals some value of S', hence of S.

A random-linear combination with a challenge θ reduces multi-column lookups to single-column: $A_{\text{c}}(X) := \sum_j \theta^{m-1-j} A_j(X)$. Variable tables (where S is itself an advice column) and range checks (where S enumerates the allowed range) are special cases.

Remark 5.2 (Active rows and blinding)**.** The lookup constraints above, like the permutation constraints of §5.3, hold on the *active* rows only: the deployed system reserves the trailing rows of every column for random blinding values (§5.9). Each product-update constraint is therefore multiplied by the factor $\big(1-(\ell_{\text{last}}(X)+\ell_{\text{blind}}(X))\big)$, where ℓ_{blind} and ℓ_{last} are Lagrange-basis selectors of the blinding rows and of the boundary row immediately before them — computed directly from the domain by both parties, not committed circuit columns — and boundary constraints such as $\ell_{\text{last}}(X)\big(Z(X)^2 - Z(X)\big) = 0$ pin the running product to $\{0,1\}$ at the boundary row. The random rows consequently neither violate the arguments nor weaken them.

5.5 The inner-product polynomial commitment

The Halo 2 polynomial commitment, instantiating the abstract PCS of the *Crypto Guide*, is the inner-product argument of Bootle–Cerulli–Chaidos–Groth–Petit (EUROCRYPT 2016) as refined in Bulletproofs (Bünz et al., 2017). We give the construction in detail. Write $d := 2^k$ for the degree bound (equivalently, the number of generators); the round count $k = \log_2 d$ reuses the letter k of Definition 5.1, harmlessly, because in the assembled protocol of §5.7 the committed polynomials have degree $< n$, so $d = n = 2^k$ there. The letter H likewise does double duty, inherited from two literatures: $H \subset \mathbb{F}$ is the PLONK evaluation domain (as in Z_H), while $H \in \mathbb{G}$ is the Pedersen blinding generator of the *Crypto Guide*; the ambient set ($\mathbb{F}$ versus $\mathbb{G}$) disambiguates every occurrence. Two further letters carry double duty: p denotes the group order (as in $\mathbb{F}_p$) and, with an argument or in $\deg p$, the committed polynomial $p(X)$; and the permutation polynomials $s_i(X) \in \mathbb{F}[X]$ of §5.3 are distinct from the IPA structured scalars $s_i \in \mathbb{F}_p$ of §5.5.2. Type and argument disambiguate every occurrence.

Construction 5.3 (IPA-based polynomial commitment)**.** Let $\mathbb{G}$ be a cyclic group of prime order p in which DL is hard. Choose $d = 2^k$ and sample, via hash-to-curve, $\vec{G} = (G_1, \ldots, G_d)$ and $H \in \mathbb{G}$ with no known non-trivial linear relation among them.

For $p(X) = \sum_{i=0}^{d-1} a_i X^i \in \mathbb{F}_p[X]$ and blinder $r \in \mathbb{F}_p$,

$$\mathsf{Commit}(p; r) := \langle \vec{a},\, \vec{G} \rangle + [r]H \in \mathbb{G}.$$

To open p at $x \in \mathbb{F}_p$ with claimed value v, the relation is

$$\mathcal{R}_{\text{IPA}} := \Big\{ \big((P, x, v); (\vec{a}, r)\big) : P = \langle \vec{a},\, \vec{G} \rangle + [r]H,\ v = \langle \vec{a},\, \vec{b}(x) \rangle,\ \deg p \le d-1 \Big\},$$

with $\vec{b}(x) := (1, x, x^2, \ldots, x^{d-1})$.

5.5.1 The recursive halving protocol

The argument runs in $k = \log_2 d$ rounds. Initialise $\vec{a}^{(k)} := \vec{a}$, $\vec{G}^{(k)} := \vec{G}$, $\vec{b}^{(k)} := \vec{b}(x)$, and let $U \in \mathbb{G}$ be a challenge group element (in Fiat–Shamir, derived from the transcript). The prover absorbs $[v]U$ so that the relation becomes

$$P + [v]U = \langle \vec{a}, \vec{G} \rangle + [r]H + [\langle \vec{a}, \vec{b} \rangle]U.$$

In round j (descending from k to 1): split each vector into halves and let

$$L_j := \langle \vec{a}^{(j)}_{\mathrm{lo}}, \vec{G}^{(j)}_{\mathrm{hi}} \rangle + [\ell_j]H + [\langle \vec{a}^{(j)}_{\mathrm{lo}}, \vec{b}^{(j)}_{\mathrm{hi}} \rangle]U,$$
$$R_j := \langle \vec{a}^{(j)}_{\mathrm{hi}}, \vec{G}^{(j)}_{\mathrm{lo}} \rangle + [r_j]H + [\langle \vec{a}^{(j)}_{\mathrm{hi}}, \vec{b}^{(j)}_{\mathrm{lo}} \rangle]U.$$

The verifier returns a uniform $u_j \in \mathbb{F}_p^*$. Both parties fold:

$$\vec{a}^{(j-1)} := u_j\, \vec{a}^{(j)}_{\mathrm{lo}} + u_j^{-1}\, \vec{a}^{(j)}_{\mathrm{hi}},$$
$$\vec{G}^{(j-1)} := u_j^{-1}\, \vec{G}^{(j)}_{\mathrm{lo}} + u_j\, \vec{G}^{(j)}_{\mathrm{hi}},$$
$$\vec{b}^{(j-1)} := u_j^{-1}\, \vec{b}^{(j)}_{\mathrm{lo}} + u_j\, \vec{b}^{(j)}_{\mathrm{hi}}.$$

After k rounds each vector has length 1. The prover sends the final scalar $a := \vec{a}^{(0)}$ and an aggregated blinder r^*. The verifier accepts iff

$$P + [v]U + \sum_{j=1}^{k} \big([u_j^2]L_j + [u_j^{-2}]R_j \big) \;\stackrel{?}{=}\; [a]\, G^{(0)} + [r^*]H + [a \cdot b^{(0)}]U, \tag{5}$$

where $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ and the challenges determine $\vec{s}$ as below.

5.5.2 The structured scalars and the polynomial g

Write the binary expansion $i = \sum_{\ell=0}^{k-1} b_\ell 2^\ell$ for $i \in [0, d)$. By induction on the folding,

$$G^{(0)} = \langle \vec{s}, \vec{G} \rangle, \qquad b^{(0)} = \langle \vec{s}, \vec{b}(x) \rangle = g(x; u_1, \ldots, u_k),$$

where

$$s_i := \prod_{\ell=0}^{k-1} u_{\ell+1}^{(-1)^{1-b_\ell}}, \qquad g(X; u_1, \ldots, u_k) := \prod_{\ell=1}^{k} \big(u_\ell^{-1} + u_\ell X^{2^{\ell-1}} \big).$$

The essential asymmetry is the following: $g(X)$ has degree $d-1$ and the verifier can evaluate it at x in $O(\log d)$ field operations; but computing $G^{(0)} = \mathsf{Commit}(g; 0)$ requires an $O(d)$-size multi-scalar multiplication (MSM). This asymmetry is the seed of the accumulation scheme (§5.6).

Remark 5.4 (The deployed IPA normalisation)**.** The deployed opening (`halo2_proofs` 0.3.2, `poly/commitment/prover.rs` and `verifier.rs`) uses the rescaled Halo convention, not the Bulletproofs one above: the folding weights are $(1, u_j^{-1})$ on the coefficient vector and $(1, u_j)$ on $\vec{b}$ and $\vec{G}$, the verifier accumulates $[u_j^{-1}]L_j + [u_j]R_j$, and the deferred polynomial is $g(X) = \prod_{i=0}^{k-1} \big(1 + u_{k-1-i} X^{2^i}\big)$. The opening also begins with an extra blinding commitment S and two challenges ξ, z, and ends with two scalars: the collapsed coefficient c and a synthetic blinder f. The two conventions are equivalent up to per-round rescaling of the folded vectors and relabelling (u_j^2 here corresponds to the code's u_j, with the roles of L_j and R_j exchanged); every statement of this section transfers.

5.5.3 Worked example: an IPA opening at $d = 4$

Take $d = 4$, so $k = 2$ rounds. Let $p(X) = a_0 + a_1X + a_2X^2 + a_3X^3$ with $\vec{a} = (a_0, a_1, a_2, a_3)$. The verifier holds $P = \langle \vec{a}, \vec{G} \rangle + [r]H$ (we suppress r to declutter; the H-part propagates identically) and requires $v = p(x)$ at some $x \in \mathbb{F}_p$.

Set $\vec{b}^{(2)} := (1, x, x^2, x^3)$ and $\vec{G}^{(2)} := \vec{G}$. After the prover absorbs $[v]U$, the relation is $P + [v]U = \langle \vec{a}, \vec{G} \rangle + \langle \vec{a}, \vec{b}^{(2)} \rangle U$.

Round 2. Split into halves of size 2. The prover sends

$$L_2 = a_0G_3 + a_1G_4 + (a_0x^2 + a_1x^3)U,$$
$$R_2 = a_2G_1 + a_3G_2 + (a_2 + a_3x)U.$$

The verifier picks $u_2 \in \mathbb{F}_p^*$ and both parties fold to length-2 vectors as above.

Round 1. Split the length-2 vectors. The prover sends L_1, R_1 analogously; the verifier picks u_1. After folding, all three vectors have length 1.

Final check. With $i = b_0 + 2b_1$ ranging over $\{0, 1, 2, 3\}$:

$$s_0 = u_1^{-1}u_2^{-1}, \qquad s_1 = u_1u_2^{-1}, \qquad s_2 = u_1^{-1}u_2, \qquad s_3 = u_1u_2.$$

The verifier evaluates $g(x; u_1, u_2) = (u_1^{-1} + u_1x)(u_2^{-1} + u_2x^2)$ in four field multiplications (u_1x, x^2, u_2x^2, and the product of the two factors), and computes $G^{(0)} = \sum_{i=0}^{3} [s_i]G_{i+1}$ as a length-4 MSM. The check (5) becomes

$$P + [v]U + [u_1^2]L_1 + [u_1^{-2}]R_1 + [u_2^2]L_2 + [u_2^{-2}]R_2 \stackrel{?}{=} [a]G^{(0)} + [a \cdot g(x; u_1, u_2)]\, U.$$

The folding rule for $\vec{G}$ ensures that $G^{(0)}$ is a specific linear combination of the original $\vec{G}$ with coefficients $\vec{s}$; that the s_i match $\prod_\ell (u_\ell^{-1}$ or $u_\ell)$ according to the binary expansion of i is exactly the closed-form identity. The verifier can evaluate g in $O(\log d)$ field operations (about $3k - 2$: k coefficient products, $k - 1$ squarings of x, $k - 1$ factor multiplications) but recomputing $G^{(0)}$ takes $O(d)$ MSM — the asymmetry the accumulator defers.

5.5.4 Knowledge soundness via rewinding

Theorem 5.5 (Knowledge soundness of IPA)**.** *Under the DL assumption on $\mathbb{G}$, the interactive opening protocol of Construction 5.3 is knowledge-sound for $\mathcal{R}_{\mathrm{IPA}}$ with knowledge error $O(d/|\mathbb{F}_p|)$, via an expected-polynomial-time tree extractor. The Fiat–Shamir-compiled argument inherits the bound only up to the naive $q^{O(\log d)}$ random-oracle query loss; the tight non-interactive bound needs the AGM (§3).*

Sketch of the tree extractor. The extractor rewinds the prover $\mathcal{P}^*$ to obtain, at each round, *three* accepting transcripts that agree on (L_j, R_j) but carry pairwise distinct challenges u_j. The round-j verification equation involves the three monomials $u_j^{-2}, 1, u_j^2$, whose coefficients — the unknown representations of L_j, the folded commitment, and R_j over the generators — the extractor must determine; two transcripts would leave this system underdetermined. Three challenges with pairwise distinct squares ($u \neq \pm u'$; the colliding pairs are excluded below) yield three linear equations whose 3×3 Vandermonde-type determinant in u^2 is non-zero, so the extractor solves for the round-j witness halves $\vec{a}_{\mathrm{lo}}^{(j)}, \vec{a}_{\mathrm{hi}}^{(j)}$ (Lemma 3.3).

Extracting the full witness $\vec{a} \in \mathbb{F}_p^d$ requires three sibling transcripts at the deepest level, which yield one round-1 witness; three such triples, hung off pairwise distinct challenges at the next-shallower round, yield one round-2 witness; and so on, each shallower round tripling the

requirement. The tree thus has $3^k = d^{\log_2 3}$ leaves — polynomial in d (§3) — yielding polynomial expected extraction time when $d = \mathsf{poly}(\lambda)$. The theorem's knowledge error of $O(d/|\mathbb{F}_p|)$ does not come from a tree-wide union bound. Each of the $k = \log_2 d$ rounds excludes $O(1)$ bad challenges ($u_j = \pm u'_j$), contributing $O(\log d/|\mathbb{F}_p|) \le O(d/|\mathbb{F}_p|)$; Schwartz–Zippel terms of degree d account for the rest. Collisions among the resampled challenges inside the extractor's 3^k-leaf tree affect only its expected running time — the extractor rewinds and resamples — not the knowledge error; a naive union bound over the roughly 3^k sibling challenge draws would bound the tree-wide collision probability by $O(d^{\log_2 3}/|\mathbb{F}_p|)$, still negligible, but that is not the source of the theorem's figure.

This is the recursive generalisation of special soundness for Schnorr (the *Crypto Guide*): a single transcript reveals nothing, but a few transcripts on the same first message permit inversion of a small linear system; in the IPA the system is 3×3 per round and the recursion stacks k rounds. □

5.6 Accumulation and the Halo trick

The verifier's check (5) runs in $O(\log d)$ scalar operations and $\log d$ point additions, *except* for the single evaluation $G^{(0)} = \langle \vec{s}, \vec{G} \rangle$ — an MSM of length d. The accumulation idea (Bowe–Grigg–Hopwood 2019; formalised by BCMS20) is to *defer* this MSM and amortise it across many proofs.

Definition 5.6 (Atomic accumulator for IPA)**.** An IPA *accumulator instance* is a tuple $\mathsf{Acc} := (G^{(0)}, u_1, \ldots, u_k) \in \mathbb{G} \times \mathbb{F}_p^k$. It is *valid* iff $G^{(0)} = \langle \vec{s}(\vec{u}), \vec{G} \rangle = \mathsf{Commit}(g(X; u_1, \ldots, u_k); 0)$ — the deferred claim concerns the unblinded commitment, the blinder having been discharged by the $[r^*]H$ term of (5). Subsequent unary $\mathsf{Commit}(\cdot)$ abbreviates $\mathsf{Commit}(\cdot\,; 0)$.

Theorem 5.7 (Accumulation of IPA, BCMS20)**.** *There is an accumulation scheme* $(\mathsf{AccVerifier}, \mathsf{AccDecider})$ *for IPA with:*

- $\mathsf{AccVerifier}$*, on input a new IPA instance and an old accumulator, runs in* $O(\log d)$ *and outputs a new accumulator* Acc'*.*
- $\mathsf{AccDecider}$*, run* once *at the end of a chain, performs an MSM of length* d *to check* $G^{(0)} = \mathsf{Commit}(g)$*.*
- *Soundness reduces to DL on* $\mathbb{G}$ *in the random-oracle model.*

Operational interpretation. A Halo 2 verifier circuit, expressed over the scalar field of one Pasta curve and verifying a proof produced over the *other* curve, can:

1. Run the $O(\log d)$ "succinct" checks of the IPA verifier natively.
2. Witness the next-step claimed $G^{(0)\prime}$ and the next challenges $u'_1, \ldots, u'_k$ as public inputs to the next proof, deferring the MSM check.
3. Fold the deferred MSM checks: with a random challenge ρ, the two claims $G^{(0)} = \mathsf{Commit}(g)$ and $G^{(0)\prime} = \mathsf{Commit}(g')$ reduce to the single check $G^{(0)} + [\rho]\, G^{(0)\prime} = \mathsf{Commit}(g + \rho g')$, with both challenge tuples retained so that the combined length-d coefficient vector $\vec{s}(\vec{u}) + \rho\, \vec{s}(\vec{u}\,')$ is recomputable; a false claim survives the combination with probability at most $1/|\mathbb{F}_p|$. In the recursive instantiation the combined commitment is opened at a fresh random point inside the next proof's multipoint argument — each g_i there is evaluable in $O(\log d)$ — so the deferred claim emerging from the next IPA opening is again a single tuple of the form of Definition 5.6; the tuple type is closed under accumulation only through this prover-assisted step, not under verifier-side linear combination.

A single final MSM at the end of the chain validates the entire history. The Pasta cycle (§4.3) renders each step native: a Vesta-IPA proof's circuit, whose native field is the Vesta scalar field $\mathbb{F}_{p_{\mathsf{Pallas}}}$, verifies the previous Pallas-IPA proof, whose group elements (the column and quotient commitments, the (L_j, R_j) pairs, the accumulator's $G^{(0)}$) have coordinates in exactly that field; the next Pallas proof verifies the Vesta proof by the symmetric argument. This cross-curve alternation is what enables proof-carrying data without expensive non-native field simulation. The same coincidence is what lets Orchard's $\mathbb{F}_{p_{\mathsf{Pallas}}}$-native Action circuit manipulate the Pallas points cm, rk, and $\mathsf{cv}^{\mathsf{net}}$ directly (§5.8).

Orchard's deployment does not recurse: the verifier of an Action proof pays the deferred MSM itself, as part of the standard verification cost. Even there the accumulation idea is load-bearing: a Zcash node verifying many Action proofs takes a random linear combination of their deferred MSMs and evaluates the combination as one MSM, amortising the linear-time work across the whole batch — the deployed, non-recursive use of the same algebraic structure.

The mathematical core is the recognition that $G^{(0)} = \mathsf{Commit}(g(X; u_1, \ldots, u_k))$ — the linear-time check is *itself a polynomial commitment opening of a structured polynomial* and hence amenable to recursion within an IPA.

5.7 The complete Halo 2 protocol

We now assemble the pieces into the seven-phase protocol implemented in the halo2 codebase. Throughout, $\mathbb{F}$ denotes the constraint field (Orchard: $\mathbb{F}_{p_{\mathsf{Pallas}}}$), $\mathbb{G}$ the IPA group (Orchard: $\mathsf{Vesta}(\mathbb{F}_{p_{\mathsf{Vesta}}})$), and H the multiplicative subgroup of $\mathbb{F}$ of order n.

Setup (preprocessed, one-time). The circuit designer specifies columns, custom gates, lookup tables, and the equality permutation σ. Both parties derive commitments to the fixed-column polynomials $f_i(X)$, selector polynomials $q_i(X)$, permutation polynomials $s_i(X)$, and lookup-table polynomials $S_j(X)$, together with the Lagrange basis on H, the vanishing polynomial $Z_H(X) := X^n - 1$, and the IPA generators $\vec{G} \in \mathbb{G}^n$ and $H \in \mathbb{G}$. The commitments to the fixed, selector, permutation, and lookup-table polynomials, together with the domain parameters, constitute the *verifying key* vk; every part of it is recomputable from the public circuit description, which is why no trusted setup exists. To verify a proof one needs exactly three inputs: vk, the instance (public-input) values, and the proof itself.

Phase 1 — advice commitments. Both parties first absorb a digest of vk and the (unblinded) instance commitments into the transcript (§5.8), so every subsequent challenge depends on the circuit and the public input. The prover then constructs the advice column values $\vec{w}_i \in \mathbb{F}^n$ satisfying all gates and equality constraints; fills the reserved trailing rows with uniformly random field elements (the blinding rows of §5.9); interpolates each $\vec{w}_i$ to a polynomial $w_i(X)$ of degree $< n$ on H; commits via IPA-PCS with a fresh blinder; sends the commitments.

Phase 2 — lookup compression: challenge θ. The verifier (via Fiat–Shamir, here and throughout) returns $\theta \in \mathbb{F}$. For each lookup the prover compresses multi-column inputs and tables with powers of θ, computes the permuted columns A', S' (§5.4), and commits to them.

Phase 3 — running products: challenges β, γ. The verifier returns $\beta, \gamma \in \mathbb{F}$. The prover commits to the permutation running product Z_{perm} (§5.3) and to one lookup running product Z_{lookup} per lookup (§5.4). The order matters for soundness: the permuted columns A', S' must be fixed *before* the challenges β, γ that randomise the products are drawn.

Phase 4 — quotient challenge y. The verifier returns a fresh $y \in \mathbb{F}$. The prover assembles the *gate equation*: a single polynomial $C(X)$ that is a y-weighted linear combination of (i) each custom gate, (ii) permutation constraints, (iii) lookup constraints. By construction $C(X)$ vanishes on H

when all gates pass, so the quotient $h(X) := C(X)/Z_H(X) = C(X)/(X^n - 1)$ is a polynomial. Its degree, however, is bounded by $d(n-1) - n = (d-1)(n-1) - 1$ for maximum gate degree d — a gate of degree d multiplies up to d column polynomials of degree $< n$, and division by Z_H removes n — which exceeds the IPA's degree bound n. The prover therefore splits h into $\lceil(\deg h + 1)/n\rceil$ chunks $h_0, h_1, \ldots$ of degree $< n$, with $h(X) = \sum_i X^{ni} h_i(X)$, and commits to each chunk separately; the verifier recombines the chunk *commitments*, weighted by powers of x^n once the evaluation point x is drawn. The prover's FFTs correspondingly run over an *extended* domain whose size accommodates $\deg C$, a small multiple of n.

Phase 5 — evaluation challenge x. The verifier returns $x \in \mathbb{F}$. The prover evaluates every committed polynomial except the quotient chunks at x, and at the rotated points the constraints reference — ωx for the running products (and for any gate reading an adjacent row), $\omega^{-1}x$ for the permuted lookup input A' — and sends these claimed evaluations as field elements. No evaluation of h is sent: the verifier computes the expected quotient evaluation $C(x)/(x^n - 1)$ from the sent evaluations and enforces it against the collapsed chunk commitment inside the multipoint opening of phases 6–7, where the claimed evaluations themselves also bind to their commitments.

Phase 6 — multipoint opening. The claimed evaluations must now be bound to the commitments, and Halo 2 reduces all the required openings to a single IPA opening. The committed polynomials are grouped by their set of query points: most are queried at $\{x\}$ alone, while the running products are also queried at ωx (and A' at $\omega^{-1}x$), so only a handful of distinct point sets arise. Within each group, a challenge x_1 folds the polynomials into a single combination $q_i(X)$ by powers of x_1. (The letters q and f here follow the halo2 book's multipoint-opening notation and are unrelated to the selector and fixed-column polynomials of the Setup paragraph, which are themselves among the polynomials being folded; the index i now ranges over query-point sets, not columns.) For each group, both parties interpolate the low-degree polynomial $r_i(X)$ that agrees with the claimed evaluations on the group's point set; the quotient $f_i(X) := \big(q_i(X) - r_i(X)\big)/\prod_z (X - z)$, with z ranging over the group's points, is a polynomial precisely when the claimed evaluations are correct. The prover combines the f_i by powers of a second challenge x_2 into $f(X)$ and commits to it. The verifier returns a fresh evaluation point x_3, and the prover sends the evaluation of each q_i at x_3; the verifier reconstructs the expected value of $f(x_3)$ from the $q_i(x_3)$, the $r_i(x_3)$, and the vanishing factors — if each f_i is a genuine polynomial this value matches the committed f, and by Schwartz–Zippel a match at random x_3 convinces the verifier that every claimed evaluation was correct. A final challenge x_4 folds f and the q_i into one polynomial $f^*(X)$, whose expected evaluation at x_3 the verifier assembles from the same data; a single opening of f^* at x_3 then discharges every original opening at once.

Phase 7 — IPA opening. Prover and verifier execute the recursive halving IPA (§5.5) on f^*, yielding $\log_2 n$ pairs (L_j, R_j), a final scalar a, and a final blinder r^*. The verifier accepts iff (5) holds.

Accumulation (recursion). The next proof's verifier circuit *defers* the MSM check $G^{(0)} = \mathsf{Commit}(g(X; u_1, \ldots, u_k))$ at the end of phase 7 into an accumulator and folds it with prior accumulators (§5.6). A single decider at the very end validates the entire chain.

The concrete transcript. The deployed implementation instantiates Fiat–Shamir with a transcript built on the BLAKE2b hash function. Every prover message — commitments as curve points, claimed evaluations as field elements — is absorbed into the hash state in the order the protocol fixes, and each verifier challenge is squeezed from the state at its prescribed point, after everything it must depend on has been absorbed. Both parties also absorb the data they share in advance — the vk digest and the unblinded instance commitments — which therefore never

appear in the proof. The proof *is* the ordered sequence of the prover's messages; the verifier replays the absorptions and recomputes every challenge.

Costs. The prover's work is dominated by FFTs over the extended domain and by one $O(n)$-size MSM *per committed polynomial* — several tens in all: each advice column, the permuted lookup columns, the running products, the quotient chunks, and the multipoint polynomial f (typical Orchard Action circuit: $n = 2^{11}$, a few seconds on commodity hardware). The verifier performs $O(\log n)$ field and group operations per proof — recomputing the challenges, the gate-equation field evaluation, the $\log n$ rounds of phase 7 — plus the single deferred $O(n)$ MSM, amortised over a batch as in §5.6.

Proof size. Two kinds of data make up a proof. The IPA of phase 7 contributes $2\log_2 n + 1$ group elements (a blinding commitment and the pairs (L_j, R_j); Remark 5.4) plus two final scalars — for Orchard's $n = 2^{11}$, twenty-two points in the pairs; this is the only part that scales with n, and only logarithmically. Everything else scales with the *circuit structure*, independently of n: one commitment per advice column, two permuted-column commitments and one running product per lookup, the permutation-product and quotient-chunk commitments, the multipoint commitment, and one field element per claimed evaluation. Each Pasta point or field element serialises to 32 bytes. For the Orchard Action circuit the total is fixed by consensus (ZIP 225): a bundle's proof occupies $2720 + 2272 \cdot a$ bytes for a Actions — just under 5 kB for a single-Action bundle — the per-Action term covering the per-circuit commitments and evaluations, the constant term the evaluations of the verifying key's fixed-column and permutation polynomials together with the shared quotient, multipoint, and IPA data.

5.8 Instance columns and binding the public statement

Definition 5.1 lists *instance* columns alongside fixed and advice columns, but the arguments of §5.3–§5.7 never single them out. We single them out now, because what an Orchard Action proof *asserts* is precisely a statement about its instance values — the anchor, nullifier nf, net value commitment $\mathsf{cv}^{\mathsf{net}}$, randomised key rk, extracted new-note commitment cmx, and the $\mathsf{enableSpends}/\mathsf{enableOutputs}$ flags (the *Ironwood Guide*).

The instance polynomial. An instance column is a public-input column: its n entries $\vec{\pi} = (\pi_0, \ldots, \pi_{n-1}) \in \mathbb{F}^n$ are not part of the witness but both parties agree on them before verification. Like every other column it is interpolated over H in the Lagrange basis,

$$\pi(X) := \sum_{j=0}^{n-1} \pi_j \, \ell_j(X) \in \mathbb{F}[X], \qquad \deg \pi < n,$$

where $\ell_j(\omega^j) = 1$ and $\ell_j(\omega^{j'}) = 0$ otherwise (§5.1). The prover writes the public values into a handful of designated rows; all remaining π_j are 0.

Binding by copy constraints. Orchard binds the public values through the permutation argument of §5.3: the instance column participates in the permutation σ, and the circuit equality-constrains each instance cell to the advice cell carrying the corresponding in-circuit value (halo2's `constrain_instance`). A satisfying assignment therefore exists *only* when the advice cells holding the anchor, nf, $\mathsf{cv}^{\mathsf{net}}$, rk, cmx, and the flags equal the public values the verifier supplied. (An alternative design binds public inputs with a dedicated gate $q(X)\big(a(X) - \pi(X)\big) = 0$, where the selector q is active exactly on the designated rows; Orchard does not use it — the copy constraint reuses the existing permutation machinery at no extra gate degree.)

Where it enters the protocol. The verifier never takes $\pi(X)$ on trust from the prover. In the deployed IPA backend, *both* parties compute the commitment to each instance polynomial — unblinded, since the values are public, so the two computations agree — and absorb it into the transcript before the first challenge (Phase 1, §5.7); every Fiat–Shamir challenge thereby depends on the public input. Thereafter the instance column is treated like any other: its claimed evaluation $\pi(x)$ appears among the Phase 5 evaluations, enters the gate equation and the permutation expressions, and the multipoint opening of Phase 6 binds it to the instance commitment. Because the verifier computed that commitment itself, from the instance values it intends to verify against, the prover has no freedom over π: an evaluation inconsistent with the verifier's own values fails the opening argument.

Consequence. A prover who targets different public values $\vec{\pi}\,'$ must open against the instance commitment the *verifier* computes from the genuine $\vec{\pi}$, and must satisfy the copy constraints against those values. Knowledge soundness (§3) then extracts a witness for the relation instantiated at exactly those public values; a prover that does not know such a witness — whatever witnesses it holds for other instances — convinces the verifier with at most negligible probability. This binds the proof to its public statement: an Action proof that a node verifies against an anchor it accepts, a nullifier it will mark spent, and a value commitment it will sum into the net balance check (the *Ironwood Guide*) can only verify if the prover knew a witness consistent with *those* values.

5.9 Zero knowledge: hiding the witness in Halo 2

Knowledge soundness (§3) establishes that a convincing prover *knows* a witness; it remains to explain why the proof *reveals* no more than that. Halo 2 is zero-knowledge through two devices — hiding commitments and blinded polynomials — assembled into a simulator.

Hiding commitments. The polynomial commitment of §5.5 is *perfectly hiding*. The prover commits a polynomial p as

$$\mathrm{Commit}(p; r) \;=\; \langle \vec{p},\, \vec{G} \rangle + [r]\, H, \qquad r \xleftarrow{\$} \mathbb{F}_p,$$

with $\vec{p}$ the coefficient vector of p and H an independent generator of no known discrete-log relation to $\vec{G}$. The $[r]H$ term makes the commitment a *uniform* group element for every p, so a commitment by itself leaks nothing about what the prover committed (the perfect hiding of a Pedersen commitment, *Crypto Guide*). Every commitment the prover sends in phases 1–6 — the advice columns, the permuted lookup columns A', S', the permutation and lookup running products, the quotient chunks, and the multipoint polynomial f — has this form. The verifying-key and instance commitments are deliberately unblinded: they commit public data both parties recompute (§5.7, §5.8), so there is nothing to hide.

Blinding the witness polynomials. The prover eventually *opens* a commitment, and an opening reveals evaluations. The prover interpolates the advice columns over the domain H to polynomials of degree $< n$; to prevent those evaluations from leaking the witness, the prover does not use all n rows for circuit data. It reserves a small fixed number of rows at the bottom of the table — the *blinding rows*, never covered by an active gate — and fills them with uniform random field elements, at least one more than the number of times the verifier will open any single column. Its n evaluations pin down a degree-$(< n)$ polynomial, so with enough independent random values planted on the blinding rows the handful the verifier actually queries are jointly uniform and independent of the circuit data. The prover blinds the permutation and lookup running products the same way; Remark 5.2 describes how the constraint system exempts the blinding rows, so the random values never violate an argument. The quotient pieces admit no

such treatment — C fully determines them — so their commitment blinders, together with the blinding-row randomness they inherit through C, protect them instead; their single collapsed opening at x is the publicly computable value $C(x)/(x^n - 1)$. (The deployed implementation also commits a uniformly random polynomial alongside the quotient and opens it at the same point, keeping the multipoint argument zero-knowledge.)

Why the openings leak nothing. In each phase the verifier sees only (i) hiding commitments — uniform group elements — and (ii) a bounded number of evaluations of the committed polynomials at the Fiat–Shamir challenge points. The blinding rows make each such evaluation uniformly random subject only to the *public* relations the verifier itself checks; no information about the private witness passes through. The final inner-product opening (§5.5) likewise runs with a blinder, so it too hides its input.

The simulator. Formally, zero knowledge is the existence of a *simulator* that, given only the public input and *not* the witness, outputs a transcript indistinguishable from a real one. Halo 2's simulator runs the protocol "backwards," exactly as Schnorr's does (*Crypto Guide*): it chooses the evaluations and the final opening so as to satisfy the verifier's equations, then chooses hiding commitments consistent with them — possible precisely because the commitments are perfectly hiding and it may choose the blinders freely — and, non-interactively, *programs* the Fiat–Shamir oracle so the challenges come out as required. Since the commitments are perfectly hiding and a blinder protects every opening, the simulated transcript is distributed as a real one. Interactively this is honest-verifier zero knowledge; compiled through Fiat–Shamir in the random-oracle model it is a NIZK that reveals nothing beyond the truth of the statement.

The three properties together. For the relation fixed by a PLONKish circuit, Halo 2 thus delivers all three guarantees a SNARK should: *completeness* (an honest prover holding a satisfying witness always convinces the verifier), *knowledge soundness* (§3: from any convincing prover an extractor pulls a satisfying witness, so an accepted proof certifies one is *known*), and *zero knowledge* (a witness-free simulator reproduces the proof, so it leaks nothing else). It is exactly this combination — "the prover knows a satisfying witness, and the proof discloses nothing more" — on which the Orchard Action proof rests: knowledge soundness yields balance and nullifier integrity, zero knowledge yields privacy (*Ironwood Guide*).